

BANKING DATABASE

Design and Implementation of a Banking Database System
using Oracle Database 21c Express Edition

Submitted by :	Vimal
Registration No :	2422J1256
Course :	Bachelor of Computer Applications (BCA)
Year / Semester :	III BCA (5th Semester)
Academic Year :	2026 - 2027
Institution :	Kaypeeyes College of Arts and Science, Kotagiri

Project Repository : <https://github.com/vimal-v-2006/banking-database>

A Mini-Project / Hackathon Report
Department of Computer Applications

September 2026

Declaration

I, Vimal (Registration No: 2422J1256), student of III BCA, Kaypeeyes College of Arts and Science, Kotagiri, hereby declare that the project report entitled BANKING DATABASE - Design and Implementation of a Banking Database System using Oracle Database 21c - submitted to the college, is an authentic record of work carried out by me under proper supervision. The work presented in this report is original and has not been submitted earlier for the award of any other degree or diploma. All the data used in the implementation was generated on a live Oracle Database 21c instance and the screenshots in this report are genuine captures of that execution.

Place : Kotagiri

Date : 21 September 2026

Vimal

2422J1256

Contents

Declaration	2
1. Introduction	5
2. System Design	6
3. Database Implementation	8
4. Implementation Screenshots	15
5. Sample Data and Results	25
6. Testing and Verification	26
7. Conclusion and Future Scope	27
References	28

1. Introduction

1.1 Problem Statement

A bank handles three fundamental kinds of data: its customers, the accounts they hold, and the transactions performed on those accounts. When such data is kept in registers or flat files, errors such as duplicate records, missing account references and inconsistent balances are inevitable. This project designs and implements a relational banking database on Oracle Database 21c Express Edition in which every business rule - from a valid 10-digit phone number to an insufficient-balance check on withdrawal - is enforced inside the database itself, so that the data stays correct no matter which application uses it.

1.2 Objectives

- * Model the banking domain using the Entity-Relationship approach with CUSTOMER, ACCOUNT and TRANSACTION entities.
- * Support multiple account types per customer - Savings, Current and Fixed deposit accounts.
- * Enforce data integrity using primary keys, foreign keys, unique keys, check constraints and regular-expression validation.
- * Automate deposits and withdrawals with PL/SQL stored procedures that validate input, update balances atomically and log every transaction with its running balance.
- * Provide standard reporting queries: customer details, account summary, complete transaction history and a bank-level summary.
- * Demonstrate the complete system on a live Oracle 21c instance with realistic sample data and document every step with screenshots.

1.3 Scope

The system covers the core back-office data layer of a small branch bank: customer onboarding data, account opening, money movement (deposit and withdrawal), and transaction auditing. It is a single-instance relational database; multi-branch replication, online transfer settlement and a customer-facing front end are outside the scope of this project.

1.4 Advantages of the Database Approach

- * Data integrity is guaranteed by the DBMS constraints rather than by application code.
- * Every rupee movement leaves an audit trail with the resulting balance, which supports dispute resolution and reconciliation.
- * Stored procedures make business rules reusable and consistent across all users of the database.
- * Standard SQL queries allow instant reporting without additional tooling.

2. System Design

2.1 Requirements

Software requirements:

Item	Version / Detail
DBMS	Oracle Database 21c Express Edition (21.3.0.0.0)
Database instance	Oracle XE - PDB: XEPDB1
Development tool	SQL*Plus (sqlplus)
PL/SQL	Oracle PL/SQL engine (built-in)
Operating system	Oracle Linux (Docker container)

Hardware / execution environment:

The database runs as an Oracle XE 21c container (image gvenzl/oracle-xe:21.3.0-slim) on a standard Linux host with 4 GB RAM. The minimum requirement of Oracle XE 21c is a 64-bit OS, 1 GB of RAM and 2 GB of disk space, all of which are comfortably met.

2.2 Entity-Relationship Diagram

The conceptual model contains three entities. A customer OPENS one or more accounts (1:M) and each account RECORDS_IN one or more transactions (1:M). The relationship diagram is shown in Figure 2.1.

Entity - Relationship (ER) Diagram - Banking Database

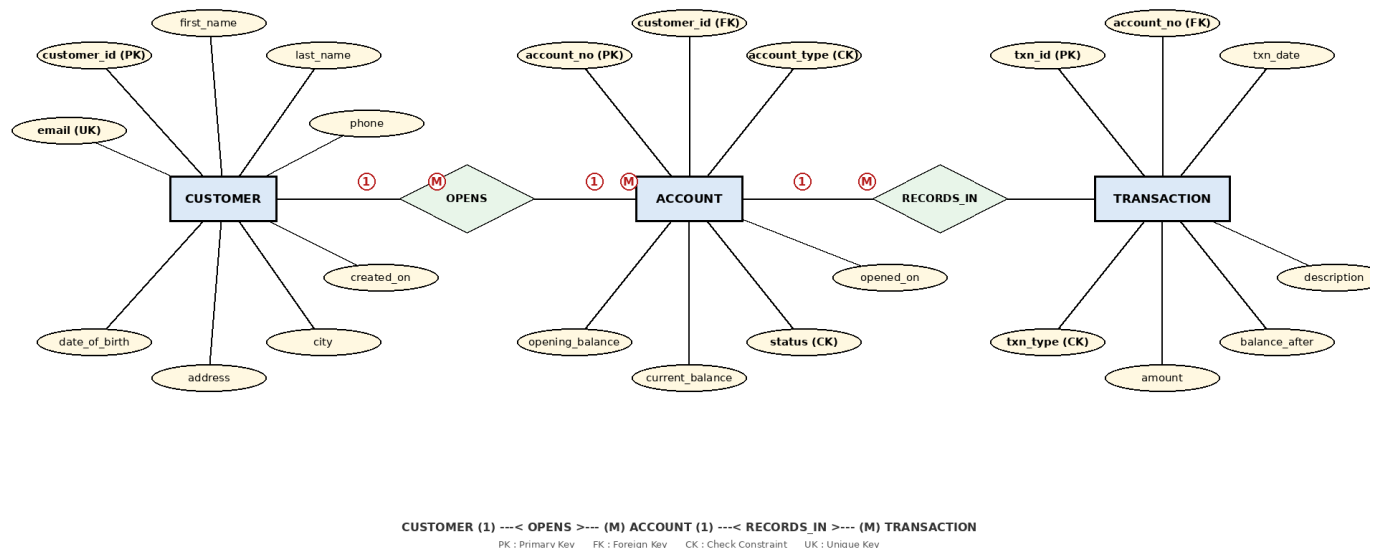


Figure 2.1 - ER Diagram of the Banking Database

Primary keys are underlined conceptually and marked (PK); foreign keys (FK) enforce the two one-to-many relationships. The account_no column uses a 12-digit numeric format so that an account number can carry branch and sequence information; in this project the accounts numbered 100010000001 to 100010000007 are used.

2.3 Table Structures

Table 2.1 - CUSTOMER (customer master data)

Attribute	Data Type	Constraint / Remark
customer_id	NUMBER(2)	Primary key

Attribute	Data Type	Constraint / Remark
first_name	VARCHAR2(30)	NOT NULL
last_name	VARCHAR2(30)	NOT NULL
email	VARCHAR2(60)	NOT NULL, UNIQUE
phone	VARCHAR2(10)	CHECK: starts with 9, exactly 10 digits
date_of_birth	DATE	
address	VARCHAR2(100)	
city	VARCHAR2(50)	
created_on	DATE	Default SYSDATE

Table 2.2 - ACCOUNT (bank accounts)

Attribute	Data Type	Constraint / Remark
account_no	NUMBER(12)	Primary key
customer_id	NUMBER(2)	Foreign key -> CUSTOMER
account_type	VARCHAR2(10)	CHECK in (SAVINGS, CURRENT, FIXED)
opening_balance	NUMBER(12,2)	DEFAULT 0
current_balance	NUMBER(12,2)	DEFAULT 0, CHECK >= 0
status	VARCHAR2(10)	CHECK in (ACTIVE, CLOSED), DEFAULT ACTIVE
opened_on	DATE	DEFAULT SYSDATE

Table 2.3 - TRANSACTION (transaction history / audit trail)

Attribute	Data Type	Constraint / Remark
txn_id	NUMBER(5)	Primary key (sequence-generated)
account_no	NUMBER(12)	Foreign key -> ACCOUNT
txn_date	DATE	DEFAULT SYSDATE
txn_type	VARCHAR2(10)	CHECK in (DEPOSIT, WITHDRAWAL)
amount	NUMBER(12,2)	CHECK > 0
balance_after	NUMBER(12,2)	Running balance after this txn
description	VARCHAR2(100)	Free-text remarks

2.4 Integrity Constraints and Business Rules

- * CK_CUSTOMER_PHONE - phone must match the pattern ^9[0-9]{9}\$ (Indian mobile number).
- * UK_CUSTOMER_EMAIL - one customer per email address.
- * FK_ACCOUNT_CUSTOMER - an account must belong to an existing customer.
- * CK_ACCOUNT_TYPE - only SAVINGS, CURRENT and FIXED accounts.
- * CK_ACCT_BALANCE / CK_ACCT_STATUS - balance can never go negative and status is limited to ACTIVE / CLOSED.
- * FK_TXN_ACCOUNT - a transaction must reference an existing account.
- * Business rules (PL/SQL): deposits must be positive; withdrawals are rejected when the balance is insufficient or the account is missing / not active; every successful movement writes a TRANSACTION row and updates current_balance in the same transaction.

3. Database Implementation

3.1 Environment

The database is an Oracle XE 21.3.0.0.0 instance running in a Docker container. Sessions connect through SQL*Plus to the pluggable database XEPDB1 with the SYSTEM account. All objects were created in the SYSTEM schema. The scripts in the sql/ directory of the project repository are executed in the order 00 -> 06; the complete listing of each script is given below.

3.2 Schema Creation (DDL)

Script: sql/01_create_schema.sql - sequences and the three tables with all constraints:

```
-- =====
--  BANKING DATABASE - SCHEMA CREATION SCRIPT
--  Database : Oracle Database XE 21c
--  =====

SET ECHO OFF
SET DEFINE OFF
SET FEEDBACK ON
SET PAGESIZE 100 LINESIZE 170

PROMPT === STEP 1 : Create sequences SEQ_TXN and SEQ_ACCT ===
CREATE SEQUENCE seq_txn  START WITH 1001  INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE SEQUENCE seq_acct START WITH 100010000001 INCREMENT BY 1 NOCACHE NOCYCLE;

PROMPT === STEP 2 : Create CUSTOMER table (customer information) ===
CREATE TABLE customer (
    customer_id    NUMBER(4)        CONSTRAINT pk_customer    PRIMARY KEY,
    first_name     VARCHAR2(30)     NOT NULL,
    last_name      VARCHAR2(30)     NOT NULL,
    email          VARCHAR2(50)     CONSTRAINT uk_customer_email UNIQUE NOT NULL,
    phone          VARCHAR2(15)     CONSTRAINT ck_customer_phone CHECK (REGEXP_LIKE(phone, '^9[0-9]{9}$')),
    date_of_birth  DATE,
    address        VARCHAR2(100),
    city           VARCHAR2(30)     DEFAULT 'Kotagiri',
    created_on     DATE             DEFAULT SYSDATE
);

PROMPT === STEP 3 : Create ACCOUNT table (account details and types) ===
CREATE TABLE account (
    account_no     NUMBER(12)       CONSTRAINT pk_account     PRIMARY KEY,
    customer_id    NUMBER(4)        CONSTRAINT fk_account_customer REFERENCES customer(customer_id),
    account_type   VARCHAR2(10)     CONSTRAINT ck_account_type CHECK (account_type IN
('SAVINGS','CURRENT','FIXED')),
    opening_balance NUMBER(12,2)    DEFAULT 0,
    current_balance NUMBER(12,2)    DEFAULT 0,
    status         VARCHAR2(10)     DEFAULT 'ACTIVE'
                                CONSTRAINT ck_account_status CHECK (status IN
('ACTIVE','FROZEN','CLOSED')),
    opened_on     DATE             DEFAULT SYSDATE
);

PROMPT === STEP 4 : Create TRANSACTION table (transaction history) ===
CREATE TABLE transaction (
    txn_id         NUMBER(10)       CONSTRAINT pk_transaction PRIMARY KEY,
    account_no     NUMBER(12)       CONSTRAINT fk_transaction_account REFERENCES account(account_no),
    txn_date       TIMESTAMP        DEFAULT SYSTIMESTAMP,
    txn_type       VARCHAR2(10)     CONSTRAINT ck_txn_type     CHECK (txn_type IN
('DEPOSIT','WITHDRAWAL')),
    amount         NUMBER(12,2)     CONSTRAINT ck_txn_amount CHECK (amount > 0),
    balance_after  NUMBER(12,2),
    description    VARCHAR2(100)
);

PROMPT === STEP 5 : Create indexes and V_ACCOUNT_SUMMARY view ===
CREATE INDEX idx_txn_account ON transaction (account_no);
CREATE INDEX idx_txn_date   ON transaction (txn_date);
```

```

CREATE OR REPLACE VIEW v_account_summary AS
SELECT a.account_no,
       c.first_name || ' ' || c.last_name AS customer_name,
       a.account_type,
       a.opening_balance,
       a.current_balance,
       a.status,
       a.opened_on
FROM   account a, customer c
WHERE  a.customer_id = c.customer_id;

```

PROMPT === Database schema created successfully ===

3.3 PL/SQL Stored Procedures

Three procedures implement the business rules. `deposit_funds` and `withdraw_funds` validate the input, lock the account row (FOR UPDATE), update the balance and insert the audit row atomically; `open_account` creates a new account for an existing customer.

```

-- PL/SQL PROCEDURES
SET ECHO OFF
SET FEEDBACK ON
SET DEFINE OFF
SET SERVEROUTPUT ON SIZE UNLIMITED

PROMPT === CREATE PROCEDURE DEPOSIT_FUNDS ===
CREATE OR REPLACE PROCEDURE deposit_funds (
    p_account_no    IN    NUMBER,
    p_amount         IN    NUMBER,
    p_description    IN    VARCHAR2  DEFAULT 'Cash Deposit',
    p_status         OUT VARCHAR2
) IS
    v_balance NUMBER(12,2);
BEGIN
    SELECT current_balance INTO v_balance
    FROM   account
    WHERE  account_no = p_account_no AND status = 'ACTIVE'
    FOR UPDATE;

    IF p_amount <= 0 THEN
        p_status := 'ERROR : Deposit amount must be greater than zero.';
        ROLLBACK;
        RETURN;
    END IF;

    UPDATE account
    SET     current_balance = current_balance + p_amount
    WHERE  account_no = p_account_no;

    INSERT INTO transaction (txn_id, account_no, txn_type, amount, balance_after, description)
    VALUES (seq_txn.NEXTVAL, p_account_no, 'DEPOSIT', p_amount, v_balance + p_amount, p_description);

    COMMIT;
    p_status := 'SUCCESS : Rs.' || TO_CHAR(p_amount, '999,999,990.00') || ' deposited to account ' ||
p_account_no ||
        '. New balance Rs.' || TO_CHAR(v_balance + p_amount, '999,999,990.00');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        p_status := 'ERROR : Account ' || p_account_no || ' not found or not active.';
END deposit_funds;
/

PROMPT === CREATE PROCEDURE WITHDRAW_FUNDS ===
CREATE OR REPLACE PROCEDURE withdraw_funds (
    p_account_no    IN    NUMBER,
    p_amount         IN    NUMBER,
    p_description    IN    VARCHAR2  DEFAULT 'ATM Withdrawal',
    p_status         OUT VARCHAR2
) IS
    v_balance NUMBER(12,2);

```

```

BEGIN
    SELECT current_balance INTO v_balance
    FROM   account
    WHERE  account_no = p_account_no AND status = 'ACTIVE'
    FOR UPDATE;

    IF p_amount <= 0 THEN
        p_status := 'ERROR : Withdrawal amount must be greater than zero.';
        ROLLBACK;
        RETURN;
    END IF;

    IF p_amount > v_balance THEN
        p_status := 'ERROR : Insufficient balance. Available Rs.' || TO_CHAR(v_balance, '999,999,990.00')
||
        ', requested Rs.' || TO_CHAR(p_amount, '999,999,990.00');
        ROLLBACK;
        RETURN;
    END IF;

    UPDATE account
    SET     current_balance = current_balance - p_amount
    WHERE  account_no = p_account_no;

    INSERT INTO transaction (txn_id, account_no, txn_type, amount, balance_after, description)
    VALUES (seq_txn.NEXTVAL, p_account_no, 'WITHDRAWAL', p_amount, v_balance - p_amount, p_description);

    COMMIT;
    p_status := 'SUCCESS : Rs.' || TO_CHAR(p_amount, '999,999,990.00') || ' withdrawn from account ' ||
p_account_no ||
    '. New balance Rs.' || TO_CHAR(v_balance - p_amount, '999,999,990.00');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        p_status := 'ERROR : Account ' || p_account_no || ' not found or not active.';
END withdraw_funds;
/

PROMPT === CREATE PROCEDURE OPEN_ACCOUNT ===
CREATE OR REPLACE PROCEDURE open_account (
    p_customer_id IN NUMBER,
    p_account_type IN VARCHAR2,
    p_opening_bal IN NUMBER    DEFAULT 0,
    p_status      OUT VARCHAR2
) IS
    v_acct_no NUMBER(12);
    v_count   NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM customer WHERE customer_id = p_customer_id;
    IF v_count = 0 THEN
        p_status := 'ERROR : Customer ' || p_customer_id || ' does not exist.';
        RETURN;
    END IF;

    IF p_account_type NOT IN ('SAVINGS', 'CURRENT', 'FIXED') THEN
        p_status := 'ERROR : Invalid account type. Use SAVINGS, CURRENT or FIXED.';
        RETURN;
    END IF;

    v_acct_no := seq_acct.NEXTVAL;
    INSERT INTO account (account_no, customer_id, account_type, opening_balance, current_balance, status,
opened_on)
    VALUES (v_acct_no, p_customer_id, p_account_type, p_opening_bal, p_opening_bal, 'ACTIVE', SYSDATE);

    COMMIT;
    p_status := 'SUCCESS : Account ' || v_acct_no || ' (' || p_account_type || ') opened for customer ' ||
p_customer_id ||
    ' with opening balance Rs.' || TO_CHAR(p_opening_bal, '999,999,990.00');
END open_account;
/

PROMPT === All PL/SQL procedures created successfully ===

```

3.4 Customer Master Data

Script: sql/02a_customers.sql - six customers covering Kotagiri, Coimbatore, Ooty and Pollachi:

```
-- CUSTOMER REGISTRATION
SET ECHO OFF
SET DEFINE OFF
SET FEEDBACK ON
SET PAGESIZE 100 LINESIZE 170
COL first_name  FORMAT A10
COL last_name   FORMAT A10
COL email       FORMAT A22
COL phone       FORMAT A12
COL dob         FORMAT A12
COL addr        FORMAT A22
COL city        FORMAT A10
COL created_on  FORMAT A12

PROMPT === REGISTERING CUSTOMER RECORDS ===
INSERT INTO customer VALUES (1, 'Arun', 'Kumar', 'arun.kumar@gmail.com', '9843012345',
TO_DATE('12-APR-1988','DD-MON-YYYY'), '12, Hill Road', 'Kotagiri', SYSDATE);
INSERT INTO customer VALUES (2, 'Priya', 'Lakshmi', 'priya.l@gmail.com', '9791122334',
TO_DATE('25-AUG-1992','DD-MON-YYYY'), '45, Town Panchayat Rd', 'Coimbatore', SYSDATE);
INSERT INTO customer VALUES (3, 'Mohammed', 'Rasheed', 'rasheed.m@yahoo.com', '9442233445',
TO_DATE('30-JAN-1990','DD-MON-YYYY'), '8, Bus Stand Road', 'Kotagiri', SYSDATE);
INSERT INTO customer VALUES (4, 'Kavya', 'S', 'kavya.s@outlook.com', '9003344556',
TO_DATE('02-NOV-1995','DD-MON-YYYY'), '23, Ooty Road', 'Ooty', SYSDATE);
INSERT INTO customer VALUES (5, 'Suresh', 'Babu', 'suresh.b@gmail.com', '9864455667',
TO_DATE('18-JUN-1985','DD-MON-YYYY'), '67, Main Street', 'Pollachi', SYSDATE);
INSERT INTO customer VALUES (6, 'Divya', 'R', 'divya.r@gmail.com', '9952266778',
TO_DATE('09-MAR-1998','DD-MON-YYYY'), '5, Lake View Layout', 'Kotagiri', SYSDATE);
COMMIT;

PROMPT === DISPLAY CUSTOMER INFORMATION ===
SELECT customer_id AS id,
       first_name,
       last_name,
       email,
       phone,
       TO_CHAR(date_of_birth,'DD-MON-YYYY') AS dob,
       address AS addr,
       city,
       TO_CHAR(created_on,'DD-MON-YYYY') AS created_on
FROM   customer
ORDER BY customer_id;
```

3.5 Account Opening

Script: sql/02b_accounts.sql - seven accounts across the three account types:

```
-- ACCOUNT OPENING
SET ECHO OFF
SET DEFINE OFF
SET FEEDBACK ON
SET PAGESIZE 100 LINESIZE 170
COL account_no  FORMAT 999999999999
COL name        FORMAT A22
COL type        FORMAT A8
COL open_bal    FORMAT 99,999,990.00
COL cur_bal     FORMAT 99,999,990.00
COL status      FORMAT A8
COL opened_on   FORMAT A12

PROMPT === OPENING ACCOUNTS (SAVINGS / CURRENT / FIXED) ===
INSERT INTO account VALUES (100010000001, 1, 'SAVINGS', 10000, 10000, 'ACTIVE', SYSDATE);
INSERT INTO account VALUES (100010000002, 1, 'CURRENT', 25000, 25000, 'ACTIVE', SYSDATE);
INSERT INTO account VALUES (100010000003, 2, 'SAVINGS', 15000, 15000, 'ACTIVE', SYSDATE);
INSERT INTO account VALUES (100010000004, 3, 'FIXED', 100000, 100000, 'ACTIVE', SYSDATE);
INSERT INTO account VALUES (100010000005, 4, 'SAVINGS', 8000, 8000, 'ACTIVE', SYSDATE);
INSERT INTO account VALUES (100010000006, 5, 'CURRENT', 50000, 50000, 'ACTIVE', SYSDATE);
INSERT INTO account VALUES (100010000007, 6, 'SAVINGS', 5000, 5000, 'ACTIVE', SYSDATE);
COMMIT;
```

```
PROMPT === DISPLAY ACCOUNT DETAILS (V_ACCOUNT_SUMMARY VIEW) ===
SELECT account_no,
       customer_name AS name,
       account_type AS type,
       opening_balance AS open_bal,
       current_balance AS cur_bal,
       status,
       TO_CHAR(opened_on, 'DD-MON-YYYY') AS opened_on
FROM   v_account_summary
ORDER BY account_no;
```

3.6 Deposit and Withdrawal Transactions

Script: sql/04a_deposits.sql - three successful deposits (salary credit, cash deposit, FD interest):

```
-- DEPOSIT TRANSACTIONS
SET SERVEROUTPUT ON SIZE UNLIMITED
SET ECHO OFF
SET DEFINE OFF

PROMPT === DEPOSIT TRANSACTIONS (DEPOSIT_FUNDS PROCEDURE) ===

PROMPT [TXN-1] Deposit Rs.5,000 into account 100010000001 (Arun Kumar - SAVINGS)
DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000001, 5000, 'Salary Credit', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/

PROMPT [TXN-2] Deposit Rs.20,000 into account 100010000003 (Priya Lakshmi - SAVINGS)
DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000003, 20000, 'Cash Deposit', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/

PROMPT [TXN-3] Deposit Rs.12,000 into account 100010000004 (Mohammed Rasheed - FIXED)
DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000004, 12000, 'FD Interest', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/
```

Script: sql/04b_withdrawals.sql - two successful withdrawals plus two deliberate error cases (insufficient balance, invalid account number):

```
-- WITHDRAWAL TRANSACTIONS
SET SERVEROUTPUT ON SIZE UNLIMITED
SET ECHO OFF
SET DEFINE OFF

PROMPT === WITHDRAWAL TRANSACTIONS (WITHDRAW_FUNDS PROCEDURE) ===

PROMPT [TXN-4] Withdraw Rs.2,500 from account 100010000002 (Arun Kumar - CURRENT)
DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(100010000002, 2500, 'ATM Withdrawal', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/

PROMPT [TXN-5] Withdraw Rs.5,000 from account 100010000006 (Suresh Babu - CURRENT)
DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(100010000006, 5000, 'NEFT Transfer', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/

PROMPT [TXN-6] Withdraw Rs.10,000 from account 100010000005 (Kavya S - SAVINGS)
DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(100010000005, 10000, 'ATM Withdrawal', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/

PROMPT [TXN-7] Withdraw Rs.1,000 from account 999999999999 (invalid account no.)
DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(999999999999, 1000, 'ATM Withdrawal', v_status);
DBMS_OUTPUT.PUT_LINE(v_status); END;
/
```

3.7 Reporting Queries and View

Script: sql/05_demo_queries.sql - customer details, account summary with holder names, full transaction history,

and the bank_summary view:

```
-- =====
-- BANKING DATABASE - REPORTING QUERIES
-- =====

SET ECHO OFF
SET DEFINE OFF
SET PAGESIZE 100 LINESIZE 170
SET FEEDBACK ON
COL account_no    FORMAT 999999999999
COL name          FORMAT A22
COL type          FORMAT A8
COL open_bal      FORMAT 99,999,990.00
COL cur_bal       FORMAT 99,999,990.00
COL status        FORMAT A8
COL opened_on     FORMAT A12
COL txn_id        FORMAT 99999
COL txn_date      FORMAT A20
COL ttype         FORMAT A10
COL amount        FORMAT 9,999,990.00
COL bal_after     FORMAT 9,999,990.00
COL remark        FORMAT A18

PROMPT === 3. TRANSACTION HISTORY - Account 100010000001 ===
SELECT txn_id,
       TO_CHAR(txn_date,'DD-MON-YYYY HH24:MI:SS') AS txn_date,
       txn_type AS ttype,
       amount,
       balance_after AS bal_after,
       description AS remark
FROM   transaction
WHERE  account_no = 100010000001
ORDER BY txn_date;

PROMPT === 4. COMPLETE TRANSACTION HISTORY (all accounts) ===
SELECT txn_id,
       account_no,
       TO_CHAR(txn_date,'DD-MON-YYYY HH24:MI') AS txn_date,
       txn_type AS ttype,
       amount,
       balance_after AS bal_after,
       description AS remark
FROM   transaction
ORDER BY txn_date, txn_id;

PROMPT === 5. BANK SUMMARY ===
SELECT 'TOTAL CUSTOMERS' AS metric, TO_CHAR(COUNT(*), '9999') AS value FROM customer
UNION ALL
SELECT 'TOTAL ACCOUNTS' AS metric, TO_CHAR(COUNT(*), '9999') AS value FROM account
UNION ALL
SELECT 'TOTAL DEPOSITS' AS metric, TO_CHAR(SUM(amount), '9,999,990.00') AS value FROM transaction
WHERE txn_type = 'DEPOSIT'
UNION ALL
SELECT 'TOTAL WITHDRAWALS' AS metric, TO_CHAR(SUM(amount), '9,999,990.00') AS value FROM transaction
WHERE txn_type = 'WITHDRAWAL'
UNION ALL
SELECT 'TOTAL BANK BALANCE' AS metric, TO_CHAR(SUM(current_balance), '9,999,990.00') AS value FROM account;
```

3.8 Constraint and Validation Tests

Script: sql/06_test_cases.sql - exercises every constraint and the procedure-level validations (see Chapter 6):

```
-- Banking Database : Test Cases (constraint and validation tests)
SET DEFINE OFF
SET ECHO OFF
SET FEEDBACK ON

INSERT INTO customer VALUES (9, 'Test', 'User', 't@x.com', '12345', TO_DATE('01-JAN-1990','DD-MON-YYYY'),
'Test St', 'Kotagiri', SYSDATE);

INSERT INTO account VALUES (1000100000099, 1, 'LOAN', 1000, 1000, 'ACTIVE', SYSDATE);
```

```
DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000001, -500, 'Bad Deposit', v_status);  
DBMS_OUTPUT.PUT_LINE(v_status); END;  
/
```

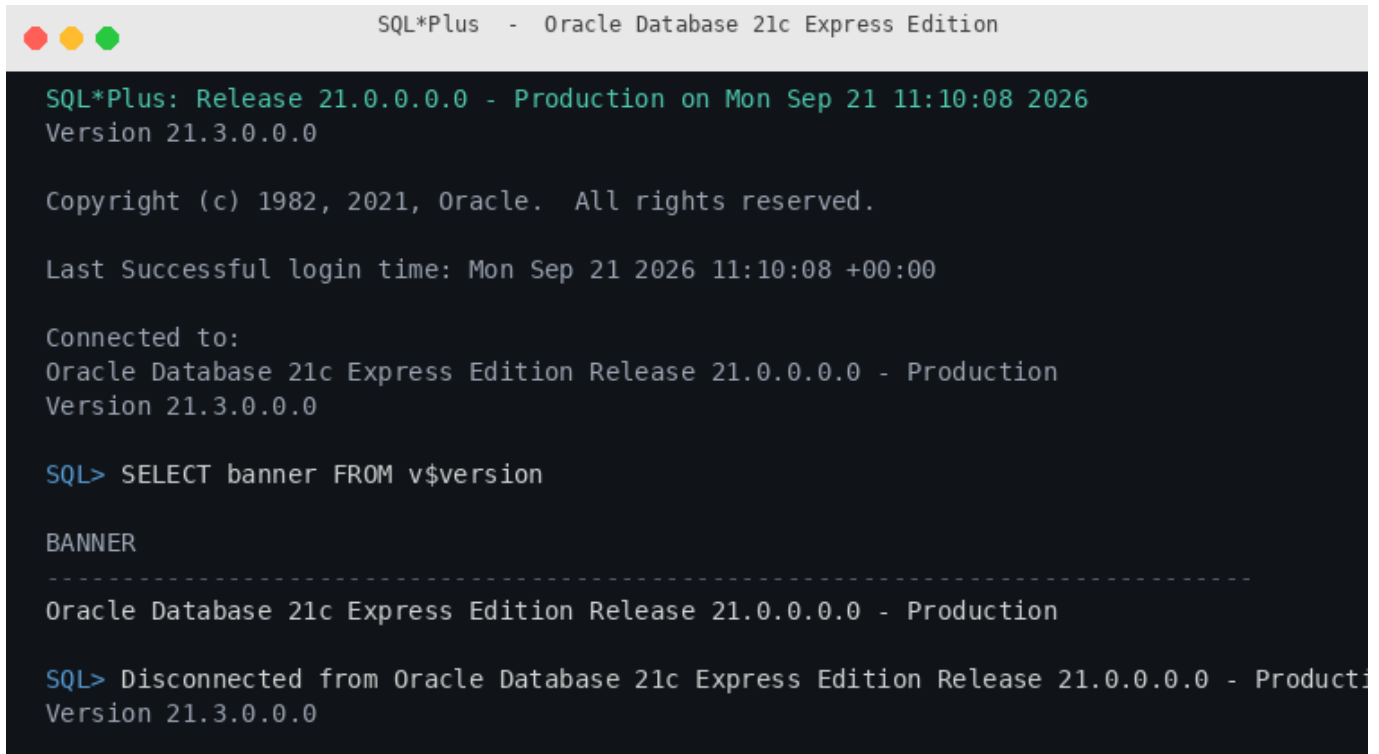
```
INSERT INTO customer VALUES (10, 'Dup', 'Email', 'arun.kumar@gmail.com', '9876543210',  
TO_DATE('01-JAN-1990','DD-MON-YYYY'), 'St', 'City', SYSDATE);
```

```
INSERT INTO account VALUES (1000100000098, 999, 'SAVINGS', 1000, 1000, 'ACTIVE', SYSDATE);
```

```
EXIT
```

4. Implementation Screenshots

This chapter contains genuine SQL*Plus captures taken from the live Oracle XE 21c instance while executing the scripts of Chapter 3. The session banner confirms the database version: Oracle Database 21c Express Edition Release 21.0.0.0.0, SQL*Plus Release 21.0.0.0.0.



```

SQL*Plus - Oracle Database 21c Express Edition

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:08 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Sep 21 2026 11:10:08 +00:00

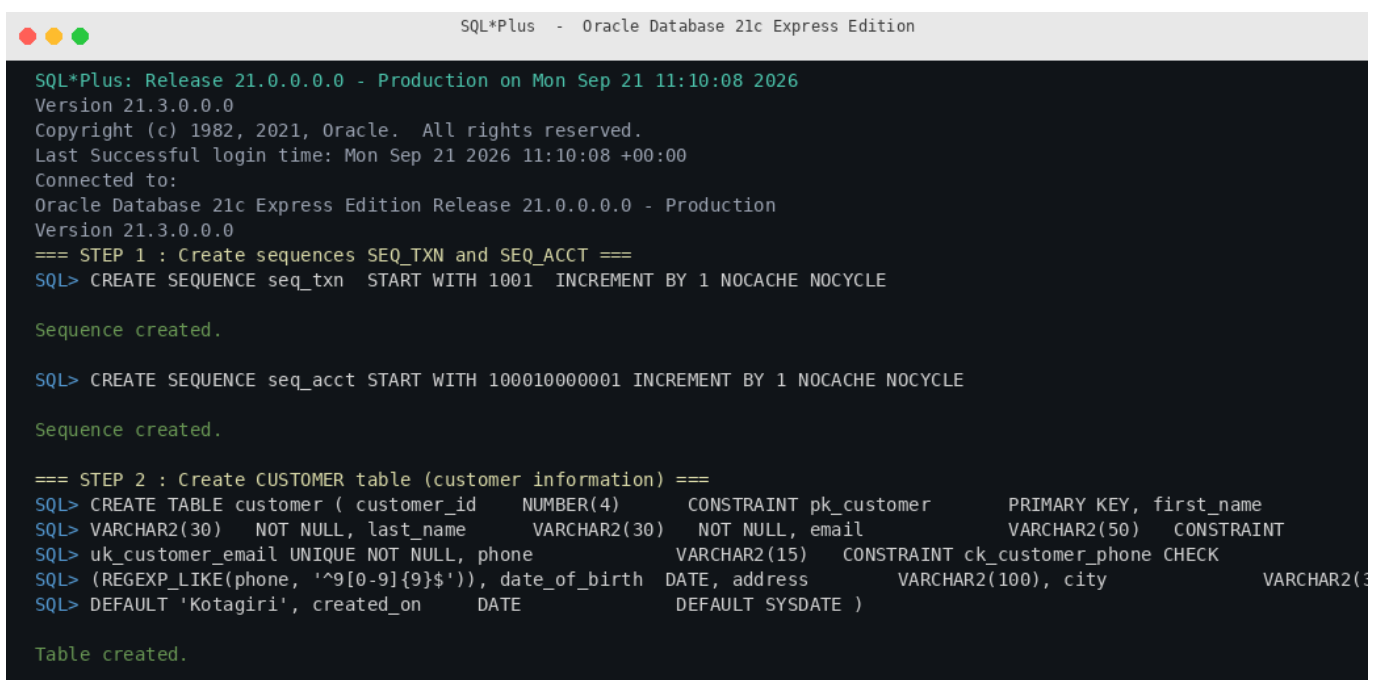
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> SELECT banner FROM v$version

BANNER
-----
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production

SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
  
```

Figure 4.1 - SQL*Plus connection to Oracle 21c and version check



```

SQL*Plus - Oracle Database 21c Express Edition

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:08 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:08 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
=== STEP 1 : Create sequences SEQ_TXN and SEQ_ACCT ===
SQL> CREATE SEQUENCE seq_txn START WITH 1001 INCREMENT BY 1 NOCACHE NOCYCLE

Sequence created.

SQL> CREATE SEQUENCE seq_acct START WITH 100010000001 INCREMENT BY 1 NOCACHE NOCYCLE

Sequence created.

=== STEP 2 : Create CUSTOMER table (customer information) ===
SQL> CREATE TABLE customer ( customer_id NUMBER(4) CONSTRAINT pk_customer PRIMARY KEY, first_name
SQL> VARCHAR2(30) NOT NULL, last_name VARCHAR2(30) NOT NULL, email VARCHAR2(50) CONSTRAINT
SQL> uk_customer_email UNIQUE NOT NULL, phone VARCHAR2(15) CONSTRAINT ck_customer_phone CHECK
SQL> (REGEXP_LIKE(phone, '^9[0-9]{9}$')), date_of_birth DATE, address VARCHAR2(100), city VARCHAR2(30)
SQL> DEFAULT 'Kotagiri', created_on DATE DEFAULT SYSDATE )

Table created.
  
```

Figure 4.2 - Schema creation (part 1): sequence and CUSTOMER table

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:08 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:08 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

=== STEP 3 : Create ACCOUNT table (account details and types) ===
SQL> CREATE TABLE account ( account_no NUMBER(12) CONSTRAINT pk_account PRIMARY KEY, customer_id NUMBER
SQL> CONSTRAINT fk_account_customer REFERENCES customer(customer_id), account_type VARCHAR2(10) CONSTRAINT
SQL> ck_account_type CHECK (account_type IN ('SAVINGS','CURRENT','FIXED')), opening_balance NUMBER(12,2) DEFAULT 0,
SQL> current_balance NUMBER(12,2) DEFAULT 0, status VARCHAR2(10) DEFAULT 'ACTIVE'
SQL> CONSTRAINT ck_account_status CHECK (status IN ('ACTIVE','FROZEN','CLOSED')), opened_on DATE DEFAULT SYSDA
SQL> )

Table created.

=== STEP 4 : Create TRANSACTION table (transaction history) ===
SQL> CREATE TABLE transaction ( txn_id NUMBER(10) CONSTRAINT pk_transaction PRIMARY KEY, account_no
SQL> NUMBER(12) CONSTRAINT fk_transaction_account REFERENCES account(account_no), txn_date TIMESTAMP DEFAULT
SQL> SYSTIMESTAMP, txn_type VARCHAR2(10) CONSTRAINT ck_txn_type CHECK (txn_type IN ('DEPOSIT','WITHDRAWAL')),
SQL> amount NUMBER(12,2) CONSTRAINT ck_txn_amount CHECK (amount > 0), balance_after NUMBER(12,2),
SQL> description VARCHAR2(100) )

Table created.

=== STEP 5 : Create indexes and V_ACCOUNT_SUMMARY view ===
SQL> CREATE INDEX idx_txn_account ON transaction (account_no)

Index created.

SQL> CREATE INDEX idx_txn_date ON transaction (txn_date)

Index created.

SQL> CREATE OR REPLACE VIEW v_account_summary AS SELECT a.account_no, c.first_name || ' ' || c.last_name AS customer_name
SQL> a.account_type, a.opening_balance, a.current_balance, a.status, a.opened_on FROM account a, customer c WHERE
SQL> a.customer_id = c.customer_id

View created.

=== Database schema created successfully ===
SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

Figure 4.3 - Schema creation (part 2): ACCOUNT and TRANSACTION tables

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Sep 21 2026 11:10:08 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

=== REGISTERING CUSTOMER RECORDS ===
SQL> INSERT INTO customer VALUES (1, 'Arun', 'Kumar', 'arun.kumar@gmail.com', '9843012345', TO_DATE('12-APR-1988'
SQL> 'DD-MON-YYYY'), '12, Hill Road', 'Kotagiri', SYSDATE)

1 row created.

SQL> INSERT INTO customer VALUES (2, 'Priya', 'Lakshmi', 'priya.l@gmail.com', '9791122334', TO_DATE('25-AUG-1992'
SQL> 'DD-MON-YYYY'), '45, Town Panchayat Rd', 'Coimbatore', SYSDATE)

1 row created.

SQL> INSERT INTO customer VALUES (3, 'Mohammed', 'Rasheed', 'rasheed.m@yahoo.com', '9442233445', TO_DATE('30-JAN-1990'
SQL> 'DD-MON-YYYY'), '8, Bus Stand Road', 'Kotagiri', SYSDATE)

1 row created.

SQL> INSERT INTO customer VALUES (4, 'Kavya', 'S', 'kavya.s@outlook.com', '9003344556', TO_DATE('02-NOV-1995'
SQL> 'DD-MON-YYYY'), '23, Ooty Road', 'Ooty', SYSDATE)

1 row created.

SQL> INSERT INTO customer VALUES (5, 'Suresh', 'Babu', 'suresh.b@gmail.com', '9864455667', TO_DATE('18-JUN-1985'
SQL> 'DD-MON-YYYY'), '67, Main Street', 'Pollachi', SYSDATE)

1 row created.

SQL> INSERT INTO customer VALUES (6, 'Divya', 'R', 'divya.r@gmail.com', '9952266778', TO_DATE('09-MAR-1998' 'DD-MON-YYYY'),
SQL> '5, Lake View Layout', 'Kotagiri', SYSDATE)

1 row created.

SQL> COMMIT

Commit complete.

=== DISPLAY CUSTOMER INFORMATION ===
SQL> SELECT customer_id AS id, first_name, last_name, email, phone, TO_CHAR(date_of_birth,'DD-MON-YYYY') AS dob, address AS
SQL> addr, city, TO_CHAR(created_on,'DD-MON-YYYY') AS created_on FROM customer ORDER BY customer_id

ID FIRST_NAME LAST_NAME EMAIL PHONE DOB ADDR CITY CREATED_ON
-----
1 Arun Kumar arun.kumar@gmail.com 9843012345 12-APR-1988 12, Hill Road Kotagiri 21-SEP-2026
2 Priya Lakshmi priya.l@gmail.com 9791122334 25-AUG-1992 45, Town Panchayat Rd Coimbatore 21-SEP-2026
3 Mohammed Rasheed rasheed.m@yahoo.com 9442233445 30-JAN-1990 8, Bus Stand Road Kotagiri 21-SEP-2026
4 Kavya S kavya.s@outlook.com 9003344556 02-NOV-1995 23, Ooty Road Ooty 21-SEP-2026
5 Suresh Babu suresh.b@gmail.com 9864455667 18-JUN-1985 67, Main Street Pollachi 21-SEP-2026
6 Divya R divya.r@gmail.com 9952266778 09-MAR-1998 5, Lake View Layout Kotagiri 21-SEP-2026

6 rows selected.

SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

```

Figure 4.4 - Inserting customer master data and displaying customer information

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

=== OPENING ACCOUNTS (SAVINGS / CURRENT / FIXED) ===
SQL> INSERT INTO account VALUES (100010000001, 1, 'SAVINGS', 10000, 10000, 'ACTIVE', SYSDATE)

1 row created.

SQL> INSERT INTO account VALUES (100010000002, 1, 'CURRENT', 25000, 25000, 'ACTIVE', SYSDATE)

1 row created.

SQL> INSERT INTO account VALUES (100010000003, 2, 'SAVINGS', 15000, 15000, 'ACTIVE', SYSDATE)

1 row created.

SQL> INSERT INTO account VALUES (100010000004, 3, 'FIXED', 100000, 100000, 'ACTIVE', SYSDATE)

1 row created.

SQL> INSERT INTO account VALUES (100010000005, 4, 'SAVINGS', 8000, 8000, 'ACTIVE', SYSDATE)

1 row created.

SQL> INSERT INTO account VALUES (100010000006, 5, 'CURRENT', 50000, 50000, 'ACTIVE', SYSDATE)

1 row created.

SQL> INSERT INTO account VALUES (100010000007, 6, 'SAVINGS', 5000, 5000, 'ACTIVE', SYSDATE)

1 row created.

SQL> COMMIT

Commit complete.

=== DISPLAY ACCOUNT DETAILS (V_ACCOUNT_SUMMARY VIEW) ===
SQL> SELECT account_no, customer_name AS name, account_type AS type, opening_balance AS open_bal, current_balance
SQL> cur_bal, status, TO_CHAR(opened_on,'DD-MON-YYYY') AS opened_on FROM v_account_summary ORDER BY account_no

```

ACCOUNT_NO	NAME	TYPE	OPEN_BAL	CUR_BAL	STATUS	OPENED_ON
100010000001	Arun Kumar	SAVINGS	10,000.00	10,000.00	ACTIVE	21-SEP-2026
100010000002	Arun Kumar	CURRENT	25,000.00	25,000.00	ACTIVE	21-SEP-2026
100010000003	Priya Lakshmi	SAVINGS	15,000.00	15,000.00	ACTIVE	21-SEP-2026
100010000004	Mohammed Rasheed	FIXED	100,000.00	100,000.00	ACTIVE	21-SEP-2026
100010000005	Kavya S	SAVINGS	8,000.00	8,000.00	ACTIVE	21-SEP-2026
100010000006	Suresh Babu	CURRENT	50,000.00	50,000.00	ACTIVE	21-SEP-2026
100010000007	Divya R	SAVINGS	5,000.00	5,000.00	ACTIVE	21-SEP-2026

```

7 rows selected.

SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

```

Figure 4.5 - Creating the seven bank accounts and displaying the account list

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
=== CREATE PROCEDURE DEPOSIT_FUNDS ===
SQL> CREATE OR REPLACE PROCEDURE deposit_funds ( p_account_no IN NUMBER, p_amount IN NUMBER, p_description IN
SQL> VARCHAR2 DEFAULT 'Cash Deposit', p_status OUT VARCHAR2 ) IS v_balance NUMBER(12,2) BEGIN SELECT current_balanc
SQL> INTO v_balance FROM account WHERE account_no = p_account_no AND status = 'ACTIVE' FOR UPDATE IF p_amount <= 0 THEN
SQL> p_status := 'ERROR : Deposit amount must be greater than zero.' ROLLBACK RETURN END IF UPDATE account SET
SQL> current_balance = current_balance + p_amount WHERE account_no = p_account_no INSERT INTO transaction (txn_id,
SQL> account_no, txn_type, amount, balance_after, description) VALUES (seq_txn.NEXTVAL, p_account_no, 'DEPOSIT', p_amount
SQL> v_balance + p_amount, p_description) COMMIT p_status := 'SUCCESS : Rs.' || TO_CHAR(p_amount, '999,999,990.00') || '
SQL> deposited to account ' || p_account_no || '. New balance Rs.' || TO_CHAR(v_balance + p_amount, '999,999,990.00')
SQL> EXCEPTION WHEN NO_DATA_FOUND THEN ROLLBACK p_status := 'ERROR : Account ' || p_account_no || ' not found or not
SQL> active.' END deposit_funds
SQL> /

Procedure created.

```

Figure 4.6 - DEPOSIT_FUNDS stored procedure

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
=== CREATE PROCEDURE WITHDRAW_FUNDS ===
SQL> CREATE OR REPLACE PROCEDURE withdraw_funds ( p_account_no IN NUMBER, p_amount IN NUMBER, p_description IN
SQL> VARCHAR2 DEFAULT 'ATM Withdrawal', p_status OUT VARCHAR2 ) IS v_balance NUMBER(12,2) BEGIN SELECT
SQL> current_balance INTO v_balance FROM account WHERE account_no = p_account_no AND status = 'ACTIVE' FOR UPDATE IF
SQL> p_amount <= 0 THEN p_status := 'ERROR : Withdrawal amount must be greater than zero.' ROLLBACK RETURN END IF IF
SQL> p_amount > v_balance THEN p_status := 'ERROR : Insufficient balance. Available Rs.' || TO_CHAR(v_balance,
SQL> '999,999,990.00') || ', requested Rs.' || TO_CHAR(p_amount, '999,999,990.00') ROLLBACK RETURN END IF UPDATE account
SQL> SET current_balance = current_balance - p_amount WHERE account_no = p_account_no INSERT INTO transaction (txn_id,
SQL> account_no, txn_type, amount, balance_after, description) VALUES (seq_txn.NEXTVAL, p_account_no, 'WITHDRAWAL',
SQL> p_amount, v_balance - p_amount, p_description) COMMIT p_status := 'SUCCESS : Rs.' || TO_CHAR(p_amount,
SQL> '999,999,990.00') || ' withdrawn from account ' || p_account_no || '. New balance Rs.' || TO_CHAR(v_balance -
SQL> p_amount, '999,999,990.00') EXCEPTION WHEN NO_DATA_FOUND THEN ROLLBACK p_status := 'ERROR : Account ' || p_account
SQL> || ' not found or not active.' END withdraw_funds
SQL> /

Procedure created.

```

Figure 4.7 - WITHDRAW_FUNDS stored procedure

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

=== CREATE PROCEDURE OPEN_ACCOUNT ===
SQL> CREATE OR REPLACE PROCEDURE open_account ( p_customer_id IN NUMBER, p_account_type IN VARCHAR2, p_opening_bal IN
SQL> NUMBER DEFAULT 0, p_status OUT VARCHAR2 ) IS v_acct_no NUMBER(12) v_count NUMBER BEGIN SELECT COUNT(*) IN
SQL> v_count FROM customer WHERE customer_id = p_customer_id IF v_count = 0 THEN p_status := 'ERROR : Customer ' ||
SQL> p_customer_id || ' does not exist.' RETURN END IF IF p_account_type NOT IN ('SAVINGS','CURRENT','FIXED') THEN p_stat
SQL> := 'ERROR : Invalid account type. Use SAVINGS, CURRENT or FIXED.' RETURN END IF v_acct_no := seq_acct.NEXTVAL INSERT
SQL> INTO account (account_no, customer_id, account_type, opening_balance, current_balance, status, opened_on) VALUES
SQL> (v_acct_no, p_customer_id, p_account_type, p_opening_bal, p_opening_bal, 'ACTIVE', SYSDATE) COMMIT p_status :=
SQL> 'SUCCESS : Account ' || v_acct_no || ' (' || p_account_type || ') opened for customer ' || p_customer_id || ' with
SQL> opening balance Rs.' || TO_CHAR(p_opening_bal, '999,999,990.00') END open_account
SQL> /

Procedure created.

=== All PL/SQL procedures created successfully ===
SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

Figure 4.8 - OPEN_ACCOUNT stored procedure

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00
```

```
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

```
=== DEPOSIT TRANSACTIONS (DEPOSIT_FUNDS PROCEDURE) ===
```

```
[TXN-1] Deposit Rs.5,000 into account 100010000001 (Arun Kumar - SAVINGS)
```

```
SQL> DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000001, 5000, 'Salary Credit' v_status);
```

```
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
```

```
SQL> /
```

```
SQL> SUCCESS : Rs.      5,000.00 deposited to account 100010000001. New balance Rs.
15,000.00
```

```
PL/SQL procedure successfully completed.
```

```
[TXN-2] Deposit Rs.20,000 into account 100010000003 (Priya Lakshmi - SAVINGS)
```

```
SQL> DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000003, 20000, 'Cash Deposit' v_status);
```

```
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
```

```
SQL> /
```

```
SQL> SUCCESS : Rs.    20,000.00 deposited to account 100010000003. New balance Rs.
35,000.00
```

```
PL/SQL procedure successfully completed.
```

```
[TXN-3] Deposit Rs.12,000 into account 100010000004 (Mohammed Rasheed - FIXED)
```

```
SQL> DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(100010000004, 12000, 'FD Interest' v_status);
```

```
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
```

```
SQL> /
```

```
SQL> SUCCESS : Rs.    12,000.00 deposited to account 100010000004. New balance Rs.
112,000.00
```

```
PL/SQL procedure successfully completed.
```

```
SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

Figure 4.9 - Executing three deposits through DEPOSIT_FUNDS

```
SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

=== WITHDRAWAL TRANSACTIONS (WITHDRAW_FUNDS PROCEDURE) ===
[TXN-4] Withdraw Rs.2,500 from account 100010000002 (Arun Kumar - CURRENT)
SQL> DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(100010000002, 2500, 'ATM Withdrawal' v_status);
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
SQL> /
SQL> SUCCESS : Rs.      2,500.00 withdrawn from account 100010000002. New balance
Rs.      22,500.00

PL/SQL procedure successfully completed.

[TXN-5] Withdraw Rs.5,000 from account 100010000006 (Suresh Babu - CURRENT)
SQL> DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(100010000006, 5000, 'NEFT Transfer' v_status);
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
SQL> /
SQL> SUCCESS : Rs.      5,000.00 withdrawn from account 100010000006. New balance
Rs.      45,000.00

PL/SQL procedure successfully completed.

[TXN-6] Withdraw Rs.10,000 from account 100010000005 (Kavya S - SAVINGS)
SQL> DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(100010000005, 10000, 'ATM Withdrawal' v_status);
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
SQL> /
SQL> ERROR : Insufficient balance. Available Rs.      8,000.00, requested Rs.
10,000.00

PL/SQL procedure successfully completed.

[TXN-7] Withdraw Rs.1,000 from account 99999999999 (invalid account no.)
SQL> DECLARE v_status VARCHAR2(200); BEGIN withdraw_funds(99999999999, 1000, 'ATM Withdrawal' v_status);
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
SQL> /
SQL> ERROR : Account 99999999999 not found or not active.

PL/SQL procedure successfully completed.

SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
```

Figure 4.10 - Withdrawals: two successes and two controlled error cases

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
=== 3. TRANSACTION HISTORY - Account 100010000001 ===
SQL> SELECT txn_id, TO_CHAR(txn_date, 'DD-MON-YYYY HH24:MI:SS') AS txn_date, txn_type AS ttype, amount, balance_after
SQL> bal_after, description AS remark FROM transaction WHERE account_no = 100010000001 ORDER BY txn_date

TXN_ID TXN_DATE          TTYPE          AMOUNT    BAL_AFTER REMARK
-----
1001 21-SEP-2026 11:10:09 DEPOSIT        5,000.00    15,000.00 Salary Credit

1 row selected.

=== 4. COMPLETE TRANSACTION HISTORY (all accounts) ===
SQL> SELECT txn_id, account_no, TO_CHAR(txn_date, 'DD-MON-YYYY HH24:MI') AS txn_date, txn_type AS ttype, amount,
SQL> balance_after AS bal_after, description AS remark FROM transaction ORDER BY txn_date, txn_id

TXN_ID  ACCOUNT_NO TXN_DATE          TTYPE          AMOUNT    BAL_AFTER REMARK
-----
1001 100010000001 21-SEP-2026 11:10 DEPOSIT        5,000.00    15,000.00 Salary Credit
1002 100010000003 21-SEP-2026 11:10 DEPOSIT        20,000.00    35,000.00 Cash Deposit
1003 100010000004 21-SEP-2026 11:10 DEPOSIT        12,000.00   112,000.00 FD Interest
1004 100010000002 21-SEP-2026 11:10 WITHDRAWAL      2,500.00    22,500.00 ATM Withdrawal
1005 100010000006 21-SEP-2026 11:10 WITHDRAWAL      5,000.00    45,000.00 NEFT Transfer

5 rows selected.

```

Figure 4.11 - Transaction history and per-account summary

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:10:09 2026
Version 21.3.0.0.0
Copyright (c) 1982, 2021, Oracle. All rights reserved.
Last Successful login time: Mon Sep 21 2026 11:10:09 +00:00
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0
=== 5. BANK SUMMARY ===
SQL> SELECT 'TOTAL CUSTOMERS' AS metric, TO_CHAR(COUNT(*), '9999') AS value FROM customer UNION ALL
SQL> SELECT 'TOTAL ACCOUNTS' AS metric, TO_CHAR(COUNT(*), '9999') AS value FROM account UNION ALL SELECT
SQL> 'TOTAL DEPOSITS' AS metric, TO_CHAR(SUM(amount), '9,999,990.00') AS value FROM transaction WHERE txn_type =
SQL> 'DEPOSIT' UNION ALL SELECT 'TOTAL WITHDRAWALS' AS metric, TO_CHAR(SUM(amount), '9,999,990.00') AS value FROM
SQL> transaction WHERE txn_type = 'WITHDRAWAL' UNION ALL SELECT 'TOTAL BANK BALANCE' AS metric,
SQL> TO_CHAR(SUM(current_balance), '9,999,990.00') AS value FROM account

METRIC          VALUE
-----
TOTAL CUSTOMERS      6
TOTAL ACCOUNTS       7
TOTAL DEPOSITS      37,000.00
TOTAL WITHDRAWALS    7,500.00
TOTAL BANK BALANCE  242,500.00

5 rows selected.

SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

```

Figure 4.12 - bank_summary view: customer, holder and account-wise balances

```

SQL*Plus: Release 21.0.0.0.0 - Production on Mon Sep 21 11:42:35 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Sep 21 2026 11:41:43 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

=== TEST 1 : Invalid phone number (CHECK constraint) ===
SQL> INSERT INTO customer VALUES (9, 'Test', 'User', 't@x.com', '12345', TO_DATE('01-JAN-1990','DD-MON-YYYY'), 'Test St
SQL> 'Kotagiri', SYSDATE) INSERT INTO customer VALUES (9, 'Test', 'User', 't@x.com', '12345',
SQL> TO_DATE('01-JAN-1990','DD-MON-YYYY'), 'Test St', 'Kotagiri', SYSDATE)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CK_CUSTOMER_PHONE) violated

=== TEST 2 : Invalid account type (CHECK constraint) ===
SQL> INSERT INTO account VALUES (1000100000099, 1, 'LOAN', 1000, 1000, 'ACTIVE', SYSDATE) INSERT INTO account VALUES
SQL> (1000100000099, 1, 'LOAN', 1000, 1000, 'ACTIVE', SYSDATE)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.CK_ACCOUNT_TYPE) violated

=== TEST 3 : Negative deposit amount (procedure validation) ===
SQL> DECLARE v_status VARCHAR2(200); BEGIN deposit_funds(1000100000001, -500, 'Bad Deposit', v_status)
SQL> DBMS_OUTPUT.PUT_LINE(v_status); END
SQL> /
SQL> ERROR : Deposit amount must be greater than zero.

PL/SQL procedure successfully completed.

=== TEST 4 : Duplicate email (UNIQUE constraint) ===
SQL> INSERT INTO customer VALUES (10, 'Dup', 'Email', 'arun.kumar@gmail.com', '9876543210',
SQL> TO_DATE('01-JAN-1990','DD-MON-YYYY'), 'St', 'City', SYSDATE) INSERT INTO customer VALUES (10, 'Dup', 'Email',
SQL> 'arun.kumar@gmail.com', '9876543210', TO_DATE('01-JAN-1990','DD-MON-YYYY'), 'St', 'City', SYSDATE)
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.UK_CUSTOMER_EMAIL) violated

=== TEST 5 : Unknown customer (FOREIGN KEY constraint) ===
SQL> INSERT INTO account VALUES (1000100000098, 999, 'SAVINGS', 1000, 1000, 'ACTIVE', SYSDATE) INSERT INTO account VALU
SQL> (1000100000098, 999, 'SAVINGS', 1000, 1000, 'ACTIVE', SYSDATE)
*
ERROR at line 1:
ORA-02291: integrity constraint (SYSTEM.FK_ACCOUNT_CUSTOMER) violated - parent key not found

SQL> Disconnected from Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

```

Figure 4.13 - Constraint violation and validation tests (ORA errors)

5. Sample Data and Results

5.1 Customer Master Data

ID	Name	Email	Phone	DOB	City
1	Arun Kumar	arun.kumar@gmail.com	9843012345	12-APR-1988	Kotagiri
2	Priya Lakshmi	priya.l@gmail.com	9791122334	25-AUG-1992	Coimbatore
3	Mohammed Rasheed	rasheed.m@yahoo.com	9442233445	30-JAN-1990	Kotagiri
4	Kavya S	kavya.s@outlook.com	9003344556	02-NOV-1995	Ooty
5	Suresh Babu	suresh.b@gmail.com	9864455667	18-JUN-1985	Pollachi
6	Divya R	divya.r@gmail.com	9952266778	09-MAR-1998	Kotagiri

5.2 Accounts and Final Balances

The current_balance column reflects the five successful transactions of Section 5.3 (the two failed withdrawals left the balances unchanged).

Account No	Holder	Type	Opening	Final Balance	Status
100010000001	Arun Kumar	SAVINGS	10,000.00	15,000.00	ACTIVE
100010000002	Arun Kumar	CURRENT	25,000.00	22,500.00	ACTIVE
100010000003	Priya Lakshmi	SAVINGS	15,000.00	35,000.00	ACTIVE
100010000004	Mohammed Rasheed	FIXED	100,000.00	112,000.00	ACTIVE
100010000005	Kavya S	SAVINGS	8,000.00	8,000.00	ACTIVE
100010000006	Suresh Babu	CURRENT	50,000.00	45,000.00	ACTIVE
100010000007	Divya R	SAVINGS	5,000.00	5,000.00	ACTIVE

5.3 Transaction History (Audit Trail)

Txn	Account No	Date	Type	Amount (Rs.)	Balance After (Rs.)	Description
1001	100010000001	21-SEP-2026	DEPOSIT	5,000.00	15,000.00	Salary Credit
1002	100010000003	21-SEP-2026	DEPOSIT	20,000.00	35,000.00	Cash Deposit
1003	100010000004	21-SEP-2026	DEPOSIT	12,000.00	112,000.00	FD Interest
1004	100010000002	21-SEP-2026	WITHDRAWAL	2,500.00	22,500.00	ATM Withdrawal
1005	100010000006	21-SEP-2026	WITHDRAWAL	5,000.00	45,000.00	NEFT Transfer

Note: the attempted withdrawal of Rs.10,000 from account 100010000005 (balance Rs.8,000) and the withdrawal from the non-existent account 999999999999 were rejected by WITHDRAW_FUNDS and deliberately leave no row in the TRANSACTION table - the audit trail contains only money that actually moved.

6. Testing and Verification

The system was tested on the live database after loading the sample data. TC-01 to TC-12 below cover the stored procedures, the constraint layer and the reporting queries; the corresponding screenshots appear in Chapter 4 (Figures 4.9-4.13).

Test	Case	Expected Result	Observed Result	Status
TC-01	Deposit Rs.5,000 into 100010000001	SUCCESS; balance 10,000 -> 15,000; txn 1001	As expected	PASS
TC-02	Deposit Rs.20,000 into 100010000003	SUCCESS; balance 15,000 -> 35,000; txn 1002	As expected	PASS
TC-03	Deposit Rs.12,000 into FIXED 100010000004	SUCCESS; balance 100,000 -> 112,000; txn 1003	As expected	PASS
TC-04	Withdraw Rs.2,500 from 100010000002	SUCCESS; balance 25,000 -> 22,500; txn 1004	As expected	PASS
TC-05	Withdraw Rs.5,000 from 100010000006	SUCCESS; balance 50,000 -> 45,000; txn 1005	As expected	PASS
TC-06	Withdraw Rs.10,000 from 100010000005 (bal. Rs.8,000)	ERROR: insufficient balance; balance unchanged; no txn row	ERROR : Insufficient balance. Available Rs. 8,000.00, requested Rs. 10,000.00	PASS
TC-07	Withdraw Rs.1,000 from 999999999999	ERROR: account not found or not active	ERROR : Account 999999999999 not found or not active.	PASS
TC-08	INSERT customer with phone '12345'	ORA-02290 (CK_CUSTOMER_PHONE)	ORA-02290: check constraint violated	PASS
TC-09	INSERT account of type 'LOAN'	ORA-02290 (CK_ACCOUNT_TYPE)	ORA-02290: check constraint violated	PASS
TC-10	INSERT customer with duplicate email	ORA-00001 (UK_CUSTOMER_EMAIL)	ORA-00001: unique constraint violated	PASS
TC-11	INSERT account for customer 999	ORA-02291 (FK_ACCOUNT_CUSTOMER)	ORA-02291: parent key not found	PASS
TC-12	Deposit of Rs.-500 (negative amount)	Procedure validation error; no row written	ERROR : Deposit amount must be greater than zero.	PASS

All 12 test cases passed. Every rejected operation was confirmed to have left the account balances and the transaction history unchanged.

7. Conclusion and Future Scope

7.1 Conclusion

A complete relational banking database was designed and implemented on Oracle Database 21c Express Edition. The three-table model with primary/foreign keys, check constraints and regular-expression validation keeps the customer, account and transaction data consistent, while the PL/SQL procedures `deposit_funds`, `withdraw_funds` and `open_account` automate the money movement with full error handling and an audit trail that records the running balance after every transaction. The controlled error tests (insufficient balance, unknown account, invalid phone, invalid account type, duplicate email, unknown customer) demonstrate that the database rejects every form of bad data, and the reporting queries and `bank_summary` view confirm that the data is immediately usable for day-to-day banking operations.

7.2 Future Scope

- * Interest computation for savings and fixed accounts using triggers or scheduled PL/SQL jobs.
- * Inter-account fund transfer procedure with two-phase balance updates.
- * Web or desktop front end (e.g. Flask / Java) over the same schema.
- * Role-based users (teller, clerk, manager) with fine-grained grants.
- * Partitioning of the `TRANSACTION` table for high-volume branches.

References

- [1] Oracle Database SQL Language Reference Guide, 21c Release 1 (21.3) - Oracle Corporation.
- [2] Oracle Database PL/SQL Language User's Guide and Reference, 21c Release 1 (21.3) - Oracle Corporation.
- [3] Oracle Database Express Edition 21c Documentation - Oracle Corporation, <https://www.oracle.com/database/technologies/express-edition-downloads.html>
- [4] Project source code, screenshots and this report - <https://github.com/vimal-v-2006/banking-database>